

Yapılan Çalışmalar

- **DBC ve PCAN-View İncelemesi:** PCAN-View'ın ham veri gösterdiğini, sinyal isimlerini ve birimlerini otomatik çözemediğini gördüm. Bu nedenle sinyal bazlı işlemler için Python altyapısına geçtim.
- **Mesaj ve Sinyal Kodlama:** Vector CANdb++ ile oluşturulan `my_j1939_dbc.dbc` dosyasındaki CCVS1 ve ACC1_2A mesajlarını inceledim. `cantools` ve `python-can` kütüphanelerini kullanarak hız, fren, mesafe ve cruise control sinyallerini fiziksel değerleriyle tanımlayıp 8 baytlık CAN mesajına dönüştüren `main.py` kodunu yazdım.
- **Canlı İzleme Betiği:** Hatta basılan mesajları yakalayıp DBC üzerinden çözerek terminalde sinyal isimleri ve birimleriyle anlık gösteren `monitor.py` dosyasını hazırladım.
- **Taşınabilir Ortam Kurulumu:** Projeyi GitHub'dan indiren birinin manuel ayar yapmadan doğrudan çalıştırabilmesi için `requirements.txt`, sanal ortam (`venv`) ve `PCANBasic.dll` dosyasını proje dizinine ekledim. Süreci tek tıkla otomatikleştiren `run_sender.bat` ve `run_monitor.bat` dosyalarını oluşturdum.

Kazanımlar ve Çözülen Sorunlar

- Sanal ortam kurulumunda komutları tek tek çalıştırarak yarım kalma hatasını giderdim.
- DBC kurallarına göre Extended ID (29-bit) formatında periyodik veri basmayı ve gelen veriyi canlı decode etmeyi uyguladım.
- Proje dizininde `PCANBasic.dll` ve `.bat` dosyaları kullanarak kodun farklı bilgisayarlarda bağımlılık sorunu olmadan çalışmasını sağladım.